Structure of C Program

A C program basically consists of the following parts:

- Preprocessor Commands
- Functions
- Variables
- Statements & Expressions
- Comments

Structure of C Program

```
#include <stdio.h>

int main()
{
    /* my first program in C */
    printf("ld! \n");

    return 0;
}
```

Preprocessor Commands

Main Function

Comment

Library Function

End of Program

Preprocessor Commands

- These commands tells the compiler to do preprocessing before doing actual compilation.
- Like **#include <stdio.h>** is a preprocessor command which tells a C compiler to include **stdio.h** file before going to actual compilation.

Functions

- Functions are main building blocks of any C Program.
- Every C Program will have one or more functions and **there is one mandatory function which is called main() function.**

- The C Programming language provides a set of built-in functions.
- In the above example printf() is a C built-in function which is used to print anything on the screen.

Comments

- Comments are used to give additional useful information inside a C Program.
- All the comments will be put inside /*...*/ as given in the example above.
- A comment can span through multiple lines.

```
/* comment for multiple line */
```

```
// single line comment
```

Variable and Variable Declaration

- Variable is a named memory location that can hold various values.
- All variables must be declared before they can be used.
- When we declare a variable, we tell the compiler what type of variable is being used.
- A declaration associates a group of variables with a specific data type.

Structure of C Program

```
#include <stdio.h>

int main()
{
    /* my first program in C */
    printf("Hello! \n");

    return 0;
}
```

Preprocessor Commands

Main Function

Comment

Library Function

End of Program

C’s Basic Data Type

Type	Keyword	format Specifier	Memory Requirements
Character data	char	`%c`	1 Byte
Signed whole numbers	int	`%d`	2 or 4 Byte
Floating-point numbers	float	`%f`	4 Byte
Double-precision floating-point number	double	`%lf`	8 Byte
valueless	void		---

How to Declare Variables

- To declare a variable, use this general form:

  type var-name;

- In C, a variable declaration is a statement and it must end in a semicolon (;

```c
#include<stdio.h>

int main()
{
    char ch;
    int i;
    float f;
    double d;
```

Variable

- Variables consist of **letters and digits, in any order**, except that the first character must be a letter.
- Both upper-and lowercase letters are permitted, though common usage favors the use of lowercase letters for most types of variables.
- Upper- and lowercase letters are not interchangeable (i.e., an uppercase letter is not equivalent to the corresponding lowercase letter.)

Variable (cont.)

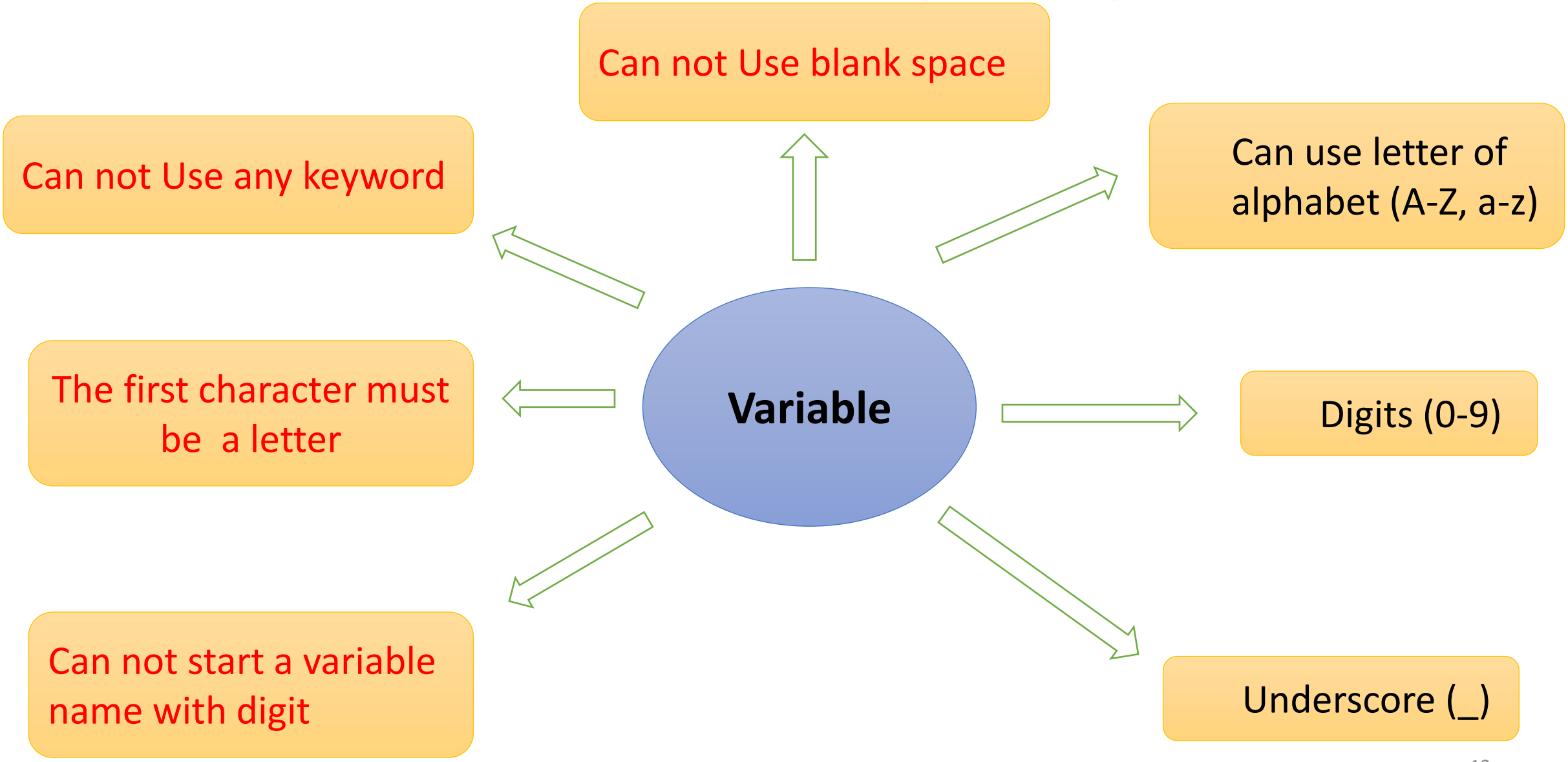
- The underscore character (_) can also be included, and is considered to be a letter.
- An underscore is often used in the middle of an variable.
- A variable may also begin with an underscore, though this is rarely done in practice.

Variable (cont.)

- Case-sensitive
- **COUNT** and **count** are not same

- As a rule, an identifier should contain enough characters so that its meaning is readily apparent.
- On the other hand, an excessive number of characters should be avoided.

Variable (Cont.)



Is it Valid Variable Name?

Apon	✓	1joty	✗
apon	✓	joty-5	✗
apon123	✓	this_is_a_long_name	✓
_sojeb_1	✓	VaReNdRa	✓

Is it Valid Variable Name?

4th	✗	The first character must be letter
“x”	✗	Illegal characters (“)
Order-no	✗	Illegal characters (-)
My variable	✗	Illegal characters (blank space)

Identifiers

- Identifiers are names that are given to various program elements, such as variables, functions and arrays.

Keywords

- There are certain reserved words, called Keywords, that have standard, predefined meaning in C
- Can be used only for their intended purpose
- Can't use as identifiers

Keywords

The standard keywords are

auto	extern	sizeof
break	floatn	static
case	for	struct
char	goto	switch

Some compilers may also include some or all of the following keywords.

ada	far	near
asm	fortran	pascal
entry	huge	

Expressions

- An expression is a combination of operators and operands.
- C expressions follow the rule of algebra

Expression	Operator
Arithmetic Expression	**+, -, *, /, %**
Logical Expression	**AND, OR, NOT**
Relational	**==, !=, <, >, <=, >=**

Assign value to variable

- To assign a value to a variable, put its name to the left of an equal sign (=).
- Put the variable you want to give the variable to the right of the equal sign.
- It is a statement, so end with a ‘;’

Edit View Search Project Build Debug wxSmith Tools Plugins Settings Help

Build target:

Start here | main.c

```c
#include<sto
#include<sto

int main()
{
    char ch;
    int i;
    float f;
    double d;

    ch = 'X';
    i = 1000;
    f = 100.12345;
    d = 123.123456789012;

    printf("%c\n", ch);
    printf("%d\n",i);
    printf("%f\n", f);
    printf("%lf", d);

    return 0;
}
```

"E:\C Code\Teach Yourself C\Chapter 01 - Fundamentals\variables\main.exe"

```
X
1000
100.123451
123.123457
Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.
```

WINDOWS-1252 | Line 1, Column 1 | Insert | Read/Write | default

Input Numbers From Keyboard

- There are several methods
- The easiest is – **scanf()** function.

- Input from Keyboard

Build Debug wxSmith Tools Plugins Settings Help

main() : int

Start here | *comment.c

```c
/* This program will take two input
from the user and print their sum.*/

#include <stdio.h>  //Library file access

int main()          //Main Function heading
{
    int num1, num2; //Variable declare

    printf("Enter first number: ");
    scanf("%d", &num1);                 //Input 1st number
    printf("Enter second number: ");
    scanf("%d", &num2);                 //Input 2nd number

    printf("Sum = %d\n\n",num1+num2);   //Print output

    return 0;
}
```

Escape Sequences

- There are certain characters in C when they are preceded by a backslash (\) they will have special meaning.
- They are used to represent like newline (\n) or tab (\t).

Escape Sequences

Escape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits

Defining Constants

- There are two simple ways in C to define constants:
  - Using ***#define*** *preprocessor.*
  - Using ***const*** *keyword.*

Using #define preprocessor

- Following is the form to use #define preprocessor to define a constant:

```
#define identifier value
```

- Symbolic constants are usually defined at the beginning of a program.

- Name of the constant
- Can be any valid identifier name

- Constant value
- Can be character/numeric/string

Using #define preprocessor

```
#define identifier value
```

```
#define MAX 10
```

```
#define PI 3.142
```

```
#define NEWLINE '\n'
```

```
#define ANSWER 'Y'
```

```
#define CR "ANABI"
```

main.c - Code::Blocks 10.05

File Edit View Search Project Build Debug wxSmith Tools Plugins Settings Help

Build target:

Management

Projects Symbols Resou

Workspace

Open files list

comment.c

main.c

Start here *comment.c main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b, sum;

    a = 10;
    b = 10;

    sum = a+b;

    printf("The summetion is %d\n\n\n",sum);

    return 0;
}
```

Table 2-1 The ASCII Character Set

ASCII Value	*Character*	*ASCII Value*	*Character*	*ASCII Value*	*Character*	*ASCII Value*	*Character*
0	NUL	32	(blank)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	–	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

The first 32 characters and the last character are control characters. Usually, they are not displayed. However, some versions of C (some computers) support special graphics characters for these ASCII values. For example, 001 may represent the character ☺, 002 may represent ☻, and so on.